

📌 Общая структура каждого варианта (подробно)

Для **каждого из 6 вариантов** студент должен создать **один файл с расширением .py** (например, `variant_1_medicines.py`). В этом файле **обязательно** должны быть реализованы следующие компоненты — в указанном порядке или логически сгруппированно.

1. Пользовательские исключения (2 шт.)

- Создайте **два класса исключений**, наследующих от `Exception`.
- Называйте их осмысленно: `Invalid...Error`, `OutOf...Error`, `Negative...Error` и т.д.
- Исключения должны вызываться при нарушении бизнес-правил (например, отрицательная цена, недопустимый возраст, пустое имя).
- Пример:

```
class InvalidPriceError(Exception):  
    pass  
  
class OutOfStockError(Exception):  
    pass
```

2. Базовый класс

- Называется обобщённо: `Medicine`, `Item`, `BankAccount`, `Athlete`, `Vehicle`, `Course`.
- Содержит **защищённые атрибуты** (с префиксом `_`), например: `_name`, `_price`, `_age`.
- Реализует:
 - **Конструктор `__init__`**, который принимает базовые параметры и **выполняет валидацию через статические методы**.
 - **Свойства с `@property`**:
 - Как минимум **два свойства**.
 - Одно — **только для чтения** (например, `name`).
 - Второе — **с геттером и сеттером**, в сеттере — **валидация с выбросом пользовательского исключения** при ошибке.
 - **Статический метод `@staticmethod`** — как минимум один, для валидации (например, `validate_price(value) → bool`).
 - **Метод класса `@classmethod`** — **ровно один**, с именем `from_string(cls, data_str)`. Он должен разбирать строку определённого формата (указанного в варианте) и возвращать новый объект.
 - **Полиморфный метод** — обычный метод **без реализации логики** в базовом классе (можно оставить `pass` или `raise NotImplementedError`), но **обязательно переопределяемый** в дочерних классах.
 - **Магические методы**:
 - `__str__(self)` — возвращает **человекочитаемую строку** с ключевой информацией об объекте.

- `__eq__(self, other)` — возвращает `True`, если два объекта **равны по определённому критерию** (указанному в варианте: цена, возраст, баланс и т.д.).
 - `__lt__(self, other)` — возвращает `True`, если текущий объект **меньше другого** по заданному критерию (для сортировки).
-

3. Дочерние классы (3 шт.)

- Каждый наследуется от базового класса: `class ChildClass(BaseClass):`
 - В конструкторе (`__init__`) вызывают `super().__init__(...)` для инициализации базовых атрибутов.
 - Добавляют **свои уникальные атрибуты** (например, `strength`, `damage`, `course_days`).
 - **Переопределяют полиморфный метод** из базового класса, реализуя **уникальную логику** для своего типа.
 - Атрибуты могут быть публичными или защищёнными — по усмотрению студента, но **должны использоваться в методах**.
-

4. Функция-демонстрация (одна на файл)

- Называется, например, `run_demo()` или `test_system()`.
- Выполняет следующие шаги **последовательно**:

а) Создание объектов

- Создаёт **4 объекта**:
 - 2 объекта — **через обычный конструктор** (`ClassName(...)`).
 - 2 объекта — **через альтернативный конструктор** (`ClassName.from_string(...)`).
- Данные для объектов — **конкретные**, указанные в варианте (например, "Парацетамол", 50, 100).

б) Тестирование ошибок

- Пытается создать **2 некорректных объекта**, которые должны вызвать **пользовательские исключения**.
- Каждая попытка обернута в `try...except`.
- При возникновении исключения — выводит сообщение:
✗ Ошибка: {e} (где `e` — текст исключения).

в) Работа со списком объектов

- Помещает 4 корректных объекта в список: `items = [obj1, obj2, obj3, obj4]`.
- **Выводит каждый объект** через `print()` (автоматически вызывается `__str__`).
- **Сортирует список** с помощью `sorted(items)` — должен сработать `__lt__`.
- **Проверяет наличие дубликатов** по критерию из `__eq__`:

```
has_duplicates = any(items[i] == items[j]
                      for i in range(len(items))
```

```
for j in range(i + 1, len(items)))
```

d) Вывод результатов

- После сортировки — снова выводит все объекты.
- Выводит:
 - «Самый [младший/дорогой/новый/высокооплачиваемый] объект: ...» (берёт последний из отсортированного списка).
 - «Есть объекты с одинаковым [критерием]: True/False».

e) Вызов функции

- Внизу файла, под всем кодом, добавьте:

```
if __name__ == "__main__":  
    run_demo()
```

5. Требования к оформлению

- **Без `input()`** — всё работает автоматически.
- **Без внешних библиотек** — только стандартный Python.
- **Код читаемый**, с отступами, без мёртвого кода.
- **Все компоненты реализованы** — иначе задание не засчитывается.



Вариант 1: Система учёта медицинских препаратов



Сценарий

Вы разрабатываете систему для аптеки. В базе — разные типы лекарств. Каждое имеет название, цену, остаток на складе и специфические характеристики.



Что реализовать

1. Исключения

```
class InvalidPriceError(Exception): ...  
class OutOfStockError(Exception): ...
```

2. Базовый класс `Medicine`

- Атрибуты: `_name`, `_price`, `_stock`
- `@property`:
 - `name` — только чтение
 - `price` — геттер/сеттер с валидацией ≥ 10
 - `stock` — геттер/сеттер с валидацией ≥ 0
- `@staticmethod validate_price(p) → p >= 10`
- `@staticmethod validate_stock(s) → s >= 0`
- `@classmethod from_string(cls, s) →` формат "Название - Цена - Остаток"
- Полиморфный метод: `get_usage_info()` → в дочерних — разный текст
- Магические методы:
 - `__str__()` → "Парацетамол: 50 руб., остаток 100"
 - `__eq__` → равны, если цена одинаковая
 - `__lt__` → дешевле, если цена меньше

3. Дочерние классы

- `Painkiller` → дополнительно: `_strength` (1–10), `get_usage_info()` → "Сила действия: {strength}"
- `Antibiotic` → дополнительно: `_course_days` (1–14), `get_usage_info()` → "Курс: {course_days} дней"
- `Vitamin` → дополнительно: `_dosage_mg` (≥ 1), `get_usage_info()` → "Дозировка: {dosage_mg} мг"

💡 В дочерних классах в `__init__` вызывайте `super().__init__(name, price, stock)` и устанавливайте свои атрибуты.

4. Демонстрация

```
meds = [
    Painkiller("Парацетамол", 50, 100, strength=7),
    Antibiotic("Амоксициллин", 200, 50, course_days=7),
    Vitamin.from_string("Витамин С - 100 - 200 - 500"), # последнее — дозировка
    Painkiller.from_string("Ибупрофен - 80 - 60 - 9")
]

# Протестируйте ошибки:
# Painkiller("Тест", 5, 10, 5) → InvalidPriceError
# Vitamin("C", 100, -5, 100) → OutOfStockError

# Выведите все через print()
# Отсортируйте по цене
# Проверьте, есть ли препараты с одинаковой ценой
```

📌 Общая структура каждого варианта (подробно)

Для **каждого из 6 вариантов** студент должен создать **один файл с расширением .py** (например, `variant_1_medicines.py`). В этом файле **обязательно** должны быть реализованы следующие компоненты — в указанном порядке или логически сгруппированно.

1. Пользовательские исключения (2 шт.)

- Создайте **два класса исключений**, наследующих от `Exception`.
- Называйте их осмысленно: `Invalid...Error`, `OutOf...Error`, `Negative...Error` и т.д.
- Исключения должны вызываться при нарушении бизнес-правил (например, отрицательная цена, недопустимый возраст, пустое имя).
- Пример:

```
class InvalidPriceError(Exception):  
    pass  
  
class OutOfStockError(Exception):  
    pass
```

2. Базовый класс

- Называется обобщённо: `Medicine`, `Item`, `BankAccount`, `Athlete`, `Vehicle`, `Course`.
- Содержит **защищённые атрибуты** (с префиксом `_`), например: `_name`, `_price`, `_age`.
- Реализует:
 - **Конструктор `__init__`**, который принимает базовые параметры и **выполняет валидацию через статические методы**.
 - **Свойства с `@property`**:
 - Как минимум **два свойства**.
 - Одно — **только для чтения** (например, `name`).
 - Второе — **с геттером и сеттером**, в сеттере — **валидация с выбросом пользовательского исключения** при ошибке.
 - **Статический метод `@staticmethod`** — как минимум один, для валидации (например, `validate_price(value) → bool`).
 - **Метод класса `@classmethod`** — **ровно один**, с именем `from_string(cls, data_str)`. Он должен разбирать строку определённого формата (указанного в варианте) и возвращать новый объект.
 - **Полиморфный метод** — обычный метод **без реализации логики** в базовом классе (можно оставить `pass` или `raise NotImplementedError`), но **обязательно переопределяемый** в дочерних классах.
 - **Магические методы**:
 - `__str__(self)` — возвращает **человекочитаемую строку** с ключевой информацией об объекте.

- `__eq__(self, other)` — возвращает `True`, если два объекта **равны по определённому критерию** (указанному в варианте: цена, возраст, баланс и т.д.).
 - `__lt__(self, other)` — возвращает `True`, если текущий объект **меньше другого** по заданному критерию (для сортировки).
-

3. Дочерние классы (3 шт.)

- Каждый наследуется от базового класса: `class ChildClass(BaseClass):`
 - В конструкторе (`__init__`) вызывают `super().__init__(...)` для инициализации базовых атрибутов.
 - Добавляют **свои уникальные атрибуты** (например, `strength`, `damage`, `course_days`).
 - **Переопределяют полиморфный метод** из базового класса, реализуя **уникальную логику** для своего типа.
 - Атрибуты могут быть публичными или защищёнными — по усмотрению студента, но **должны использоваться в методах**.
-

4. Функция-демонстрация (одна на файл)

- Называется, например, `run_demo()` или `test_system()`.
- Выполняет следующие шаги **последовательно**:

а) Создание объектов

- Создаёт **4 объекта**:
 - 2 объекта — **через обычный конструктор** (`ClassName(...)`).
 - 2 объекта — **через альтернативный конструктор** (`ClassName.from_string(...)`).
- Данные для объектов — **конкретные**, указанные в варианте (например, "Парацетамол", 50, 100).

б) Тестирование ошибок

- Пытается создать **2 некорректных объекта**, которые должны вызвать **пользовательские исключения**.
- Каждая попытка обернута в `try...except`.
- При возникновении исключения — выводит сообщение:
✗ Ошибка: {e} (где `e` — текст исключения).

в) Работа со списком объектов

- Помещает 4 корректных объекта в список: `items = [obj1, obj2, obj3, obj4]`.
- **Выводит каждый объект** через `print()` (автоматически вызывается `__str__`).
- **Сортирует список** с помощью `sorted(items)` — должен сработать `__lt__`.
- **Проверяет наличие дубликатов** по критерию из `__eq__`:

```
has_duplicates = any(items[i] == items[j]
                      for i in range(len(items))
```

```
for j in range(i + 1, len(items)))
```

d) Вывод результатов

- После сортировки — снова выводит все объекты.
- Выводит:
 - «Самый [младший/дорогой/новый/высокооплачиваемый] объект: ...» (берёт последний из отсортированного списка).
 - «Есть объекты с одинаковым [критерием]: True/False».

е) Вызов функции

- Внизу файла, под всем кодом, добавьте:

```
if __name__ == "__main__":  
    run_demo()
```

5. Требования к оформлению

- **Без `input()`** — всё работает автоматически.
- **Без внешних библиотек** — только стандартный Python.
- **Код читаемый**, с отступами, без мёртвого кода.
- **Все компоненты реализованы** — иначе задание не засчитывается.



Вариант 2: Система учёта игровых предметов



Сценарий

Игровой движок хранит предметы: оружие, броню, зелья. У каждого — название, редкость и параметры.



Что реализовать

1. Исключения

```
class InvalidRarityError(Exception): ...  
class InvalidStatError(Exception): ...
```

2. Базовый класс `Item`

- Атрибуты: `_name`, `_rarity` ("обычный", "редкий", "эпический")

- `@property`:
 - `name` — только чтение
 - `rarity` — геттер/сеттер; валидация: только из списка
- `@staticmethod validate_rarity(r) → r in ["обычный", "редкий", "эпический"]`
- `@classmethod from_string(cls, s) → "Меч - редкий - 50"`
- Полиморфный метод: `describe()` → уникальное описание
- Магические методы:
 - `__str__()` → "Меч (редкий)"
 - `__eq__` → одинаковая редкость
 - `__lt__` → "обычный" < "редкий" < "эпический" (используйте словарь рангов)

3. Дочерние классы

- `Weapon` → `_damage` (≥ 1), `describe()` → "Наносит {damage} урона"
- `Armor` → `_defense` (≥ 1), `describe()` → "Даёт {defense} защиты"
- `Potion` → `_effect` (строка), `describe()` → "Эффект: {effect}"

4. Демонстрация

```
items = [
    Weapon("Меч", "редкий", damage=50),
    Armor("Щит", "обычный", defense=30),
    Potion.from_string("Зелье HP - эпический - Восстановить 100 HP"),
    Weapon.from_string("Кинжал - обычный - 20")
]

# Ошибки:
# Weapon("Тест", "легендарный", 10) → InvalidRarityError
# Armor("Щит", "редкий", -5) → InvalidStatError

# Выведите все
# Отсортируйте по редкости
# Проверьте, есть ли два "эпических" предмета
```


📌 Общая структура каждого варианта (подробно)

Для **каждого из 6 вариантов** студент должен создать **один файл с расширением .py** (например, `variant_1_medicines.py`). В этом файле **обязательно** должны быть реализованы следующие компоненты — в указанном порядке или логически сгруппированно.

1. Пользовательские исключения (2 шт.)

- Создайте **два класса исключений**, наследующих от `Exception`.
- Называйте их осмысленно: `Invalid...Error`, `OutOf...Error`, `Negative...Error` и т.д.
- Исключения должны вызываться при нарушении бизнес-правил (например, отрицательная цена, недопустимый возраст, пустое имя).
- Пример:

```
class InvalidPriceError(Exception):  
    pass  
  
class OutOfStockError(Exception):  
    pass
```

2. Базовый класс

- Называется обобщённо: `Medicine`, `Item`, `BankAccount`, `Athlete`, `Vehicle`, `Course`.
- Содержит **защищённые атрибуты** (с префиксом `_`), например: `_name`, `_price`, `_age`.
- Реализует:
 - **Конструктор `__init__`**, который принимает базовые параметры и **выполняет валидацию через статические методы**.
 - **Свойства с `@property`**:
 - Как минимум **два свойства**.
 - Одно — **только для чтения** (например, `name`).
 - Второе — **с геттером и сеттером**, в сеттере — **валидация с выбросом пользовательского исключения** при ошибке.
 - **Статический метод `@staticmethod`** — как минимум один, для валидации (например, `validate_price(value) → bool`).
 - **Метод класса `@classmethod`** — **ровно один**, с именем `from_string(cls, data_str)`. Он должен разбирать строку определённого формата (указанного в варианте) и возвращать новый объект.
 - **Полиморфный метод** — обычный метод **без реализации логики** в базовом классе (можно оставить `pass` или `raise NotImplementedError`), но **обязательно переопределяемый** в дочерних классах.
 - **Магические методы**:
 - `__str__(self)` — возвращает **человекочитаемую строку** с ключевой информацией об объекте.

- `__eq__(self, other)` — возвращает `True`, если два объекта **равны по определённому критерию** (указанному в варианте: цена, возраст, баланс и т.д.).
 - `__lt__(self, other)` — возвращает `True`, если текущий объект **меньше другого** по заданному критерию (для сортировки).
-

3. Дочерние классы (3 шт.)

- Каждый наследуется от базового класса: `class ChildClass(BaseClass):`
 - В конструкторе (`__init__`) вызывают `super().__init__(...)` для инициализации базовых атрибутов.
 - Добавляют **свои уникальные атрибуты** (например, `strength`, `damage`, `course_days`).
 - **Переопределяют полиморфный метод** из базового класса, реализуя **уникальную логику** для своего типа.
 - Атрибуты могут быть публичными или защищёнными — по усмотрению студента, но **должны использоваться в методах**.
-

4. Функция-демонстрация (одна на файл)

- Называется, например, `run_demo()` или `test_system()`.
- Выполняет следующие шаги **последовательно**:

а) Создание объектов

- Создаёт **4 объекта**:
 - 2 объекта — **через обычный конструктор** (`ClassName(...)`).
 - 2 объекта — **через альтернативный конструктор** (`ClassName.from_string(...)`).
- Данные для объектов — **конкретные**, указанные в варианте (например, "Парацетамол", 50, 100).

б) Тестирование ошибок

- Пытается создать **2 некорректных объекта**, которые должны вызвать **пользовательские исключения**.
- Каждая попытка обернута в `try...except`.
- При возникновении исключения — выводит сообщение:
✗ Ошибка: {e} (где `e` — текст исключения).

в) Работа со списком объектов

- Помещает 4 корректных объекта в список: `items = [obj1, obj2, obj3, obj4]`.
- **Выводит каждый объект** через `print()` (автоматически вызывается `__str__`).
- **Сортирует список** с помощью `sorted(items)` — должен сработать `__lt__`.
- **Проверяет наличие дубликатов** по критерию из `__eq__`:

```
has_duplicates = any(items[i] == items[j]
                      for i in range(len(items))
```

```
for j in range(i + 1, len(items)))
```

d) Вывод результатов

- После сортировки — снова выводит все объекты.
- Выводит:
 - «Самый [младший/дорогой/новый/высокооплачиваемый] объект: ...» (берёт последний из отсортированного списка).
 - «Есть объекты с одинаковым [критерием]: True/False».

e) Вызов функции

- Внизу файла, под всем кодом, добавьте:

```
if __name__ == "__main__":  
    run_demo()
```

5. Требования к оформлению

- **Без `input()`** — всё работает автоматически.
- **Без внешних библиотек** — только стандартный Python.
- **Код читаемый**, с отступами, без мёртвого кода.
- **Все компоненты реализованы** — иначе задание не засчитывается.



Вариант 3: Система учёта банковских счетов



Сценарий

Банк предлагает разные типы счетов. У всех — номер, владелец, баланс. У каждого — особенности.



Что реализовать

1. Исключения

```
class InvalidAccountNumberError(Exception): ...  
class InsufficientFundsError(Exception): ...
```

2. Базовый класс `BankAccount`

- Атрибуты: `_account_number` (10 цифр), `_owner`, `_balance`
- `@property`:

- `account_number` — только чтение
- `balance` — геттер/сеттер; ≥ -10000 (для всех)
- `@staticmethod validate_account_number(s) → len(s)==10 and s.isdigit()`
- `@classmethod from_string(cls, s) → "ACC12345678 - Иван - 10000"`
- Полиморфный метод: `calculate_monthly_interest()` → разный %
- Магические методы:
 - `__str__()` → "Счёт ACC123...: 10000 руб."
 - `__eq__` → одинаковый баланс
 - `__lt__` → меньше баланс

3. Дочерние классы

- `CurrentAccount` → `calculate_monthly_interest()` → 0
- `SavingsAccount` → `calculate_monthly_interest()` → `balance * 0.004` (0.4% в месяц)
- `CreditAccount` → лимит -50000, `calculate_monthly_interest()` → 0, но можно уходить в минус

💡 В `CreditAccount` в сеттере баланса: если < -50000 — `InsufficientFundsError`

4. Демонстрация

```
accounts = [
    SavingsAccount("SAV11111111", "Анна", 100000),
    CreditAccount("CRD22222222", "Борис", -10000),
    CurrentAccount.from_string("CUR33333333 - Виктор - 50000"),
    SavingsAccount.from_string("SAV44444444 - Глеб - 200000")
]

# Ошибки:
# CurrentAccount("123", "Иван", 1000) → InvalidAccountNumberError
# CreditAccount("CRD55555555", "Дина", -60000) → InsufficientFundsError

# Выведите все
# Отсортируйте по балансу
# Проверьте, есть ли счета с одинаковым балансом
```

📌 Общая структура каждого варианта (подробно)

Для **каждого из 6 вариантов** студент должен создать **один файл с расширением .py** (например, `variant_1_medicines.py`). В этом файле **обязательно** должны быть реализованы следующие компоненты — в указанном порядке или логически сгруппированно.

1. Пользовательские исключения (2 шт.)

- Создайте **два класса исключений**, наследующих от `Exception`.
- Называйте их осмысленно: `Invalid...Error`, `OutOf...Error`, `Negative...Error` и т.д.
- Исключения должны вызываться при нарушении бизнес-правил (например, отрицательная цена, недопустимый возраст, пустое имя).
- Пример:

```
class InvalidPriceError(Exception):  
    pass  
  
class OutOfStockError(Exception):  
    pass
```

2. Базовый класс

- Называется обобщённо: `Medicine`, `Item`, `BankAccount`, `Athlete`, `Vehicle`, `Course`.
- Содержит **защищённые атрибуты** (с префиксом `_`), например: `_name`, `_price`, `_age`.
- Реализует:
 - **Конструктор `__init__`**, который принимает базовые параметры и **выполняет валидацию через статические методы**.
 - **Свойства с `@property`**:
 - Как минимум **два свойства**.
 - Одно — **только для чтения** (например, `name`).
 - Второе — **с геттером и сеттером**, в сеттере — **валидация с выбросом пользовательского исключения** при ошибке.
 - **Статический метод `@staticmethod`** — как минимум один, для валидации (например, `validate_price(value) → bool`).
 - **Метод класса `@classmethod`** — **ровно один**, с именем `from_string(cls, data_str)`. Он должен разбирать строку определённого формата (указанного в варианте) и возвращать новый объект.
 - **Полиморфный метод** — обычный метод **без реализации логики** в базовом классе (можно оставить `pass` или `raise NotImplementedError`), но **обязательно переопределяемый** в дочерних классах.
 - **Магические методы**:
 - `__str__(self)` — возвращает **человекочитаемую строку** с ключевой информацией об объекте.

- `__eq__(self, other)` — возвращает `True`, если два объекта **равны по определённому критерию** (указанному в варианте: цена, возраст, баланс и т.д.).
 - `__lt__(self, other)` — возвращает `True`, если текущий объект **меньше другого** по заданному критерию (для сортировки).
-

3. Дочерние классы (3 шт.)

- Каждый наследуется от базового класса: `class ChildClass(BaseClass):`
 - В конструкторе (`__init__`) вызывают `super().__init__(...)` для инициализации базовых атрибутов.
 - Добавляют **свои уникальные атрибуты** (например, `strength`, `damage`, `course_days`).
 - **Переопределяют полиморфный метод** из базового класса, реализуя **уникальную логику** для своего типа.
 - Атрибуты могут быть публичными или защищёнными — по усмотрению студента, но **должны использоваться в методах**.
-

4. Функция-демонстрация (одна на файл)

- Называется, например, `run_demo()` или `test_system()`.
- Выполняет следующие шаги **последовательно**:

а) Создание объектов

- Создаёт **4 объекта**:
 - 2 объекта — **через обычный конструктор** (`ClassName(...)`).
 - 2 объекта — **через альтернативный конструктор** (`ClassName.from_string(...)`).
- Данные для объектов — **конкретные**, указанные в варианте (например, "Парацетамол", 50, 100).

б) Тестирование ошибок

- Пытается создать **2 некорректных объекта**, которые должны вызвать **пользовательские исключения**.
- Каждая попытка обернута в `try...except`.
- При возникновении исключения — выводит сообщение:
✗ Ошибка: {e} (где `e` — текст исключения).

в) Работа со списком объектов

- Помещает 4 корректных объекта в список: `items = [obj1, obj2, obj3, obj4]`.
- **Выводит каждый объект** через `print()` (автоматически вызывается `__str__`).
- **Сортирует список** с помощью `sorted(items)` — должен сработать `__lt__`.
- **Проверяет наличие дубликатов** по критерию из `__eq__`:

```
has_duplicates = any(items[i] == items[j]
                      for i in range(len(items))
```

```
for j in range(i + 1, len(items)))
```

d) Вывод результатов

- После сортировки — снова выводит все объекты.
- Выводит:
 - «Самый [младший/дорогой/новый/высокооплачиваемый] объект: ...» (берёт последний из отсортированного списка).
 - «Есть объекты с одинаковым [критерием]: True/False».

e) Вызов функции

- Внизу файла, под всем кодом, добавьте:

```
if __name__ == "__main__":  
    run_demo()
```

5. Требования к оформлению

- **Без `input()`** — всё работает автоматически.
- **Без внешних библиотек** — только стандартный Python.
- **Код читаемый**, с отступами, без мёртвого кода.
- **Все компоненты реализованы** — иначе задание не засчитывается.



Вариант 4: Система учёта спортсменов



Сценарий

Спортивный клуб ведёт учёт спортсменов разных дисциплин.



Что реализовать

1. Исключения

```
class InvalidAgeError(Exception): ...  
class InvalidTeamError(Exception): ...
```

2. Базовый класс `Athlete`

- Атрибуты: `_name`, `_age` (12–100), `_team` (непустая строка)
- `@property`:

- `name` — только чтение
- `age` — геттер/сеттер с валидацией
- `@staticmethod validate_age(a) → 12 <= a <= 100`
- `@staticmethod validate_team(t) → t.strip() != ""`
- `@classmethod from_string(cls, s) → "Анна - 25 - Олимп-24"`
- Полиморфный метод: `get_performance()` → уникальное описание
- Магические методы:
 - `__str__()` → "Анна (25 лет, Олимп-24)"
 - `__eq__` → одинаковый возраст
 - `__lt__` → младше

3. Дочерние классы

- `Runner` → `_distance_km, _time_min, get_performance()` → "Пробежал {d} км за {t} мин"
- `Swimmer` → `_style, _distance_m, get_performance()` → "Проплыл {d} м {style}"
- `Cyclist` → `_avg_speed_kmh, get_performance()` → "Средняя скорость: {s} км/ч"

4. Демонстрация

```
athletes = [
    Runner("Иван", 22, "Спартак", 10, 40),
    Swimmer("Мария", 19, "Динамо", "кроль", 100),
    Cyclist.from_string("Алекс - 28 - ЦСКА - 35.5"),
    Runner.from_string("Сергей - 24 - Локомотив - 5 - 20")
]

# Ошибки:
# Runner("Тест", 10, "Команда", 5, 20) → InvalidAgeError
# Swimmer("Анна", 20, "", "баттерфляй", 50) → InvalidTeamError

# Выведите всех
# Отсортируйте по возрасту
# Проверьте, есть ли ровесники
```


📌 Общая структура каждого варианта (подробно)

Для **каждого из 6 вариантов** студент должен создать **один файл с расширением .py** (например, `variant_1_medicines.py`). В этом файле **обязательно** должны быть реализованы следующие компоненты — в указанном порядке или логически сгруппированно.

1. Пользовательские исключения (2 шт.)

- Создайте **два класса исключений**, наследующих от `Exception`.
- Называйте их осмысленно: `Invalid...Error`, `OutOf...Error`, `Negative...Error` и т.д.
- Исключения должны вызываться при нарушении бизнес-правил (например, отрицательная цена, недопустимый возраст, пустое имя).
- Пример:

```
class InvalidPriceError(Exception):  
    pass  
  
class OutOfStockError(Exception):  
    pass
```

2. Базовый класс

- Называется обобщённо: `Medicine`, `Item`, `BankAccount`, `Athlete`, `Vehicle`, `Course`.
- Содержит **защищённые атрибуты** (с префиксом `_`), например: `_name`, `_price`, `_age`.
- Реализует:
 - **Конструктор `__init__`**, который принимает базовые параметры и **выполняет валидацию через статические методы**.
 - **Свойства с `@property`**:
 - Как минимум **два свойства**.
 - Одно — **только для чтения** (например, `name`).
 - Второе — **с геттером и сеттером**, в сеттере — **валидация с выбросом пользовательского исключения** при ошибке.
 - **Статический метод `@staticmethod`** — как минимум один, для валидации (например, `validate_price(value) → bool`).
 - **Метод класса `@classmethod`** — **ровно один**, с именем `from_string(cls, data_str)`. Он должен разбирать строку определённого формата (указанного в варианте) и возвращать новый объект.
 - **Полиморфный метод** — обычный метод **без реализации логики** в базовом классе (можно оставить `pass` или `raise NotImplementedError`), но **обязательно переопределяемый** в дочерних классах.
 - **Магические методы**:
 - `__str__(self)` — возвращает **человекочитаемую строку** с ключевой информацией об объекте.

- `__eq__(self, other)` — возвращает `True`, если два объекта **равны по определённому критерию** (указанному в варианте: цена, возраст, баланс и т.д.).
 - `__lt__(self, other)` — возвращает `True`, если текущий объект **меньше другого** по заданному критерию (для сортировки).
-

3. Дочерние классы (3 шт.)

- Каждый наследуется от базового класса: `class ChildClass(BaseClass):`
 - В конструкторе (`__init__`) вызывают `super().__init__(...)` для инициализации базовых атрибутов.
 - Добавляют **свои уникальные атрибуты** (например, `strength`, `damage`, `course_days`).
 - **Переопределяют полиморфный метод** из базового класса, реализуя **уникальную логику** для своего типа.
 - Атрибуты могут быть публичными или защищёнными — по усмотрению студента, но **должны использоваться в методах**.
-

4. Функция-демонстрация (одна на файл)

- Называется, например, `run_demo()` или `test_system()`.
- Выполняет следующие шаги **последовательно**:

а) Создание объектов

- Создаёт **4 объекта**:
 - 2 объекта — **через обычный конструктор** (`ClassName(...)`).
 - 2 объекта — **через альтернативный конструктор** (`ClassName.from_string(...)`).
- Данные для объектов — **конкретные**, указанные в варианте (например, "Парацетамол", 50, 100).

б) Тестирование ошибок

- Пытается создать **2 некорректных объекта**, которые должны вызвать **пользовательские исключения**.
- Каждая попытка обернута в `try...except`.
- При возникновении исключения — выводит сообщение:
✗ Ошибка: {e} (где `e` — текст исключения).

в) Работа со списком объектов

- Помещает 4 корректных объекта в список: `items = [obj1, obj2, obj3, obj4]`.
- **Выводит каждый объект** через `print()` (автоматически вызывается `__str__`).
- **Сортирует список** с помощью `sorted(items)` — должен сработать `__lt__`.
- **Проверяет наличие дубликатов** по критерию из `__eq__`:

```
has_duplicates = any(items[i] == items[j]
                      for i in range(len(items))
```

```
for j in range(i + 1, len(items)))
```

d) Вывод результатов

- После сортировки — снова выводит все объекты.
- Выводит:
 - «Самый [младший/дорогой/новый/высокооплачиваемый] объект: ...» (берёт последний из отсортированного списка).
 - «Есть объекты с одинаковым [критерием]: True/False».

е) Вызов функции

- Внизу файла, под всем кодом, добавьте:

```
if __name__ == "__main__":  
    run_demo()
```

5. Требования к оформлению

- **Без `input()`** — всё работает автоматически.
- **Без внешних библиотек** — только стандартный Python.
- **Код читаемый**, с отступами, без мёртвого кода.
- **Все компоненты реализованы** — иначе задание не засчитывается.



Вариант 5: Система учёта транспортных средств



Сценарий

Автопарк управляет разным транспортом.



Что реализовать

1. Исключения

```
class InvalidYearError(Exception): ...  
class InvalidSpecError(Exception): ...
```

2. Базовый класс `Vehicle`

- Атрибуты: `_brand`, `_model`, `_year` (1900–2025)
- `@property`:

- `brand, model` — только чтение
- `year` — геттер/сеттер с валидацией
- `@staticmethod validate_year(y) → 1900 ≤ y ≤ 2025`
- `@classmethod from_string(cls, s) → "Toyota - Camry - 2020"`
- Полиморфный метод: `get_spec()` → уникальная спецификация
- Магические методы:
 - `__str__()` → `"Toyota Camry (2020)"`
 - `__eq__` → одинаковый год
 - `__lt__` → старше (меньше год)

3. Дочерние классы

- `Car` → `_fuel_consumption_l_per_100` (≥ 0), `get_spec()` → `"Расход: {f} л/100 км"`
- `Truck` → `_max_load_tons` (≥ 1), `get_spec()` → `"Грузоподъёмность: {t} т"`
- `ElectricCar` → `_range_km` (≥ 100), `get_spec()` → `"Запас хода: {r} км"`

4. Демонстрация

```
vehicles = [  
    Car("Toyota", "Camry", 2020, 8.0),  
    Truck("Volvo", "FH", 2019, 20),  
    ElectricCar.from_string("Tesla - Model 3 - 2022 - 500"),  
    Car.from_string("Honda - Civic - 2018 - 7.5")  
]  
  
# Ошибки:  
# Car("BMW", "X5", 1899, 10) → InvalidYearError  
# Truck("MAN", "TGS", 2020, -5) → InvalidSpecError  
  
# Выведите все  
# Отсортируйте по году (от старых к новым)  
# Проверьте, есть ли машины одного года
```

📌 Общая структура каждого варианта (подробно)

Для **каждого из 6 вариантов** студент должен создать **один файл с расширением .py** (например, `variant_1_medicines.py`). В этом файле **обязательно** должны быть реализованы следующие компоненты — в указанном порядке или логически сгруппированно.

1. Пользовательские исключения (2 шт.)

- Создайте **два класса исключений**, наследующих от `Exception`.
- Называйте их осмысленно: `Invalid...Error`, `OutOf...Error`, `Negative...Error` и т.д.
- Исключения должны вызываться при нарушении бизнес-правил (например, отрицательная цена, недопустимый возраст, пустое имя).
- Пример:

```
class InvalidPriceError(Exception):  
    pass  
  
class OutOfStockError(Exception):  
    pass
```

2. Базовый класс

- Называется обобщённо: `Medicine`, `Item`, `BankAccount`, `Athlete`, `Vehicle`, `Course`.
- Содержит **защищённые атрибуты** (с префиксом `_`), например: `_name`, `_price`, `_age`.
- Реализует:
 - **Конструктор `__init__`**, который принимает базовые параметры и **выполняет валидацию через статические методы**.
 - **Свойства с `@property`**:
 - Как минимум **два свойства**.
 - Одно — **только для чтения** (например, `name`).
 - Второе — **с геттером и сеттером**, в сеттере — **валидация с выбросом пользовательского исключения** при ошибке.
 - **Статический метод `@staticmethod`** — как минимум один, для валидации (например, `validate_price(value) → bool`).
 - **Метод класса `@classmethod`** — **ровно один**, с именем `from_string(cls, data_str)`. Он должен разбирать строку определённого формата (указанного в варианте) и возвращать новый объект.
 - **Полиморфный метод** — обычный метод **без реализации логики** в базовом классе (можно оставить `pass` или `raise NotImplementedError`), но **обязательно переопределяемый** в дочерних классах.
 - **Магические методы**:
 - `__str__(self)` — возвращает **человекочитаемую строку** с ключевой информацией об объекте.

- `__eq__(self, other)` — возвращает `True`, если два объекта **равны по определённому критерию** (указанному в варианте: цена, возраст, баланс и т.д.).
 - `__lt__(self, other)` — возвращает `True`, если текущий объект **меньше другого** по заданному критерию (для сортировки).
-

3. Дочерние классы (3 шт.)

- Каждый наследуется от базового класса: `class ChildClass(BaseClass):`
 - В конструкторе (`__init__`) вызывают `super().__init__(...)` для инициализации базовых атрибутов.
 - Добавляют **свои уникальные атрибуты** (например, `strength`, `damage`, `course_days`).
 - **Переопределяют полиморфный метод** из базового класса, реализуя **уникальную логику** для своего типа.
 - Атрибуты могут быть публичными или защищёнными — по усмотрению студента, но **должны использоваться в методах**.
-

4. Функция-демонстрация (одна на файл)

- Называется, например, `run_demo()` или `test_system()`.
- Выполняет следующие шаги **последовательно**:

а) Создание объектов

- Создаёт **4 объекта**:
 - 2 объекта — **через обычный конструктор** (`ClassName(...)`).
 - 2 объекта — **через альтернативный конструктор** (`ClassName.from_string(...)`).
- Данные для объектов — **конкретные**, указанные в варианте (например, "Парацетамол", 50, 100).

б) Тестирование ошибок

- Пытается создать **2 некорректных объекта**, которые должны вызвать **пользовательские исключения**.
- Каждая попытка обернута в `try...except`.
- При возникновении исключения — выводит сообщение:
✗ Ошибка: {e} (где `e` — текст исключения).

в) Работа со списком объектов

- Помещает 4 корректных объекта в список: `items = [obj1, obj2, obj3, obj4]`.
- **Выводит каждый объект** через `print()` (автоматически вызывается `__str__`).
- **Сортирует список** с помощью `sorted(items)` — должен сработать `__lt__`.
- **Проверяет наличие дубликатов** по критерию из `__eq__`:

```
has_duplicates = any(items[i] == items[j]
                      for i in range(len(items))
```

```
for j in range(i + 1, len(items))
```

d) Вывод результатов

- После сортировки — снова выводит все объекты.
- Выводит:
 - «Самый [младший/дорогой/новый/высокооплачиваемый] объект: ...» (берёт последний из отсортированного списка).
 - «Есть объекты с одинаковым [критерием]: True/False».

е) Вызов функции

- Внизу файла, под всем кодом, добавьте:

```
if __name__ == "__main__":  
    run_demo()
```

5. Требования к оформлению

- **Без `input()`** — всё работает автоматически.
- **Без внешних библиотек** — только стандартный Python.
- **Код читаемый**, с отступами, без мёртвого кода.
- **Все компоненты реализованы** — иначе задание не засчитывается.



Вариант 6: Система учёта онлайн-курсов



Сценарий

Образовательная платформа хранит курсы разных типов.



Что реализовать

1. Исключения

```
class InvalidLevelError(Exception): ...  
class InvalidPriceError(Exception): ...
```

2. Базовый класс `Course`

- Атрибуты: `_title`, `_price` (≥ 0), `_level` ("базовый", "продвинутый")
- `@property`:

- `title` — только чтение
- `price` — геттер/сеттер с валидацией
- `@staticmethod validate_price(p) → p >= 0`
- `@staticmethod validate_level(l) → l in ["базовый", "продвинутый"]`
- `@classmethod from_string(cls, s) → "Python - 1000 - продвинутый"`
- Полиморфный метод: `get_description()` → уникальное описание
- Магические методы:
 - `__str__()` → "Python (продвинутый): 1000 руб."
 - `__eq__` → одинаковая цена
 - `__lt__` → дешевле

3. Дочерние классы

- `VideoCourse` → `_duration_hours` (≥ 1), `get_description()` → "Курс из {h} часов видео"
- `InteractiveCourse` → `_num_tasks` (≥ 10), `get_description()` → "{n} интерактивных заданий"
- `Masterclass` → `_speaker`, `get_description()` → "Мастер-класс от {s}"

4. Демонстрация

```
courses = [
    VideoCourse("Python", 2000, "базовый", 10),
    InteractiveCourse("SQL", 1500, "продвинутый", 50),
    Masterclass.from_string("Искусство презентаций - 3000 - базовый - Иван
Иванов"),
    VideoCourse.from_string("JavaScript - 2500 - продвинутый - 12")
]

# Ошибки:
# VideoCourse("Тест", -100, "базовый", 5) → InvalidPriceError
# InteractiveCourse("AI", 1000, "эксперт", 20) → InvalidLevelError

# Выведите все
# Отсортируйте по цене
# Проверьте, есть ли курсы с одинаковой ценой
```

✓ Заключение

Каждый вариант:

- **Подробно описан** — что реализовать, как должно работать
- **Одинаковой сложности**
- **Покрывает все темы ООП**
- **Имеет жизненный сценарий**
- **Готов к немедленному использованию**

Раздайте студентам разные варианты — и вы получите **60 минут тихой, продуктивной работы** и объективную проверку знаний! 🧐